

Hands-On Lab: Build a Basic Omniverse Digital Twin Connected to AI Inference

Lab Objective

By the end of this lab, you will:

- Create a **simple digital twin** environment in NVIDIA Omniverse.
 - Ingest **mock IoT sensor data** into the simulation.
 - Connect to an **AI inference model** running in NVIDIA Triton Inference Server.
 - Display real-time AI predictions directly inside the Omniverse scene.
-

Prerequisites

- **Hardware:** NVIDIA RTX GPU (desktop/laptop) or access to a cloud GPU instance (AWS G5, Azure NC series, or DGX Cloud).
 - **Software:**
 - NVIDIA Omniverse Launcher (latest version)
 - Omniverse Kit or Omniverse Create installed
 - Python 3.8+
 - NVIDIA Triton Inference Server (container from NGC)
 - Sample AI model (we'll use a simple object detection model in ONNX format)
 - **Accounts:**
 - NVIDIA Developer account (for NGC access)
 - Optional: AWS/Azure account if running in the cloud
-

Step 1 — Install NVIDIA Omniverse

1. Go to <https://www.nvidia.com/omniverse> and download **Omniverse Launcher**.
 2. Install and sign in with your **NVIDIA Developer Account**.
 3. From the **Exchange** tab, install:
 - **Omniverse Kit** (or Omniverse Create)
 - **Omniverse Nucleus** (for asset and scene management)
 - **Omniverse Isaac Sim** (optional if you want robotics capabilities)
-

Step 2 — Create a New Scene

1. Launch **Omniverse Kit / Create**.
 2. Create a new **empty scene**.
 3. Add a **3D object** to represent your physical asset—e.g., a conveyor belt, robot arm, or simple cube.
 - Use **Create → Mesh → Cube** to add a placeholder object.
 4. Name it `Machine_1` in the **Scene Graph**.
-

Step 3 — Prepare Mock IoT Sensor Data

We'll simulate live data for temperature and vibration.

1. Create a Python script named `mock_sensor.py`:

```
import time, random, json
from websocket import create_connection

ws = create_connection("ws://localhost:8000")
while True:
    data = {
        "machine_id": "Machine_1",
        "temperature": round(random.uniform(60, 90), 2),
        "vibration": round(random.uniform(0.1, 0.5), 2)
    }
    ws.send(json.dumps(data))
    time.sleep(1)
```

2. Keep this running in a terminal.
-

Step 4 — Set Up Triton Inference Server

1. Pull the Triton container from NGC:

```
docker pull nvcr.io/nvidia/tritonserver:23.08-py3
```

2. Place your AI model (ONNX or TensorRT) in a model repository folder:

```
models/
  temperature_model/
    1/model.onnx
    config.pbtxt
```

3. Run Triton:

```
docker run --gpus=all --rm -p8000:8000 -p8001:8001 -p8002:8002 \
  -v $(pwd)/models:/models nvcr.io/nvidia/tritonserver:23.08-py3 \
  tritonserver --model-repository=/models
```

Step 5 — Connect Omniverse to AI Inference

1. Inside Omniverse Kit, open **Script Editor**.
2. Create a Python extension to listen for WebSocket data and send it to Triton:

```
import json, requests, websocket

TRITON_URL = "http://localhost:8000/v2/models/temperature_model/infer"

def send_to_triton(sensor_data):
    payload = {
        "inputs": [{
            "name": "input_0",
            "shape": [1, 2],
            "datatype": "FP32",
            "data": [[sensor_data["temperature"], sensor_data["vibration"]]]
        }]
    }
    r = requests.post(TRITON_URL, json=payload)
    return r.json()["outputs"][0]["data"][0]

def on_message(ws, message):
    data = json.loads(message)
    prediction = send_to_triton(data)
    # Update object color based on prediction
    stage = omni.usd.get_context().get_stage()
    machine_prim = stage.GetPrimAtPath("/World/Machine_1")
    if prediction > 0.8:
        machine_prim.GetAttribute("displayColor").Set([(1.0, 0.0, 0.0)]) # Red
    else:
        machine_prim.GetAttribute("displayColor").Set([(0.0, 1.0, 0.0)]) # Green

ws = websocket.WebSocketApp("ws://localhost:8000", on_message=on_message)
ws.run_forever()
```

3. This script listens to sensor data, runs AI inference, and updates the digital twin's visual state.

Step 6 — Run the Simulation

1. Start **Omniverse Kit** and load your scene.
2. Run `mock_sensor.py` to simulate IoT data.
3. Ensure Triton Server is running with your AI model loaded.
4. Run the Omniverse Python extension to connect everything.
5. Watch the object change colors in real time as AI predicts health status.

Step 7 — (Optional) Deploy to the Cloud

- Instead of running Triton locally, deploy it on **AWS G5 instance** or **Azure NC series GPU VM**.

- Update the `TRITON_URL` in your script to point to the cloud endpoint.
 - Use AWS IoT Core or Azure IoT Hub for live device data ingestion.
-

Lab Completion Criteria

You have successfully:

- Created a simple Omniverse scene.
 - Streamed mock IoT data into the simulation.
 - Connected Omniverse to Triton Inference Server for AI predictions.
 - Updated the digital twin's state in real time based on AI output.
-